

TRABAJO PRÁCTICO INTEGRADOR

DOCUMENTO INTEGRAL DE GESTIÓN DEL PROYECTO

Portal Web de la Asociación Cooperadora del Hospital
Municipal "Dr. Emilio Ferreyra" (Necochea)

Materia: Gestión del Desarrollo de Software

Profesor: Fernández Carbonell, Cesar Augusto

Integrantes del Grupo:

Aramis Prieto • Kevin Nielsen • Thiago Masson • Santiago Ialungo

Institución Base: UTN Extensión Áulica Necochea

Fecha de Entrega: 19 de Junio de 2026

Documento Integral de Gestión del Proyecto

Portal Web de la Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra" (Necochea)

Asignatura: Gestión del Desarrollo de Software **Profesor:** Fernández Carbonell, Cesar Augusto
Integrantes del Grupo:

- Aramis Prieto
- Kevin Nielsen
- Thiago Masson
- Santiago Ialungo

PARTE 1: Análisis, Planificación y Justificación de Decisiones Iniciales

1. Presentación del Proyecto

1.1 Descripción General

El proyecto consiste en el diseño, desarrollo e implementación de un portal web integral, seguro y de arquitectura moderna para la **Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra"** de la ciudad de Necochea.

- **Nombre del proyecto:** Portal Web de la Asociación Cooperadora del Hospital Municipal Dr. Emilio Ferreyra.
- **Institución seleccionada:** Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra" (Necochea, Buenos Aires).
- **Problemática identificada:** Tradicionalmente, la cooperadora operaba de manera puramente manual y descentralizada. La captación de socios dependía de cobradores físicos o registros en papel propensos a pérdidas. Asimismo, la comunidad carecía de un canal digital confiable y transparente para visualizar el destino exacto de sus donaciones (ej. compra de aparatología médica u obras en salas), lo que mermaba el nivel de confianza de los vecinos de Necochea y Quequén. Finalmente, el proceso de declaración y validación de transferencias bancarias directas requería conciliaciones manuales engorrosas y no generaba ningún acuse de recibo o agradecimiento inmediato al donante.
- **Solución propuesta:** Un portal web interactivo que centraliza y digitaliza los procesos clave:

1. **Registro y autogestión de socios:** Permite a los vecinos completar sus datos (incluyendo DNI único obligatorio) para formar parte digitalmente del Libro Registro de Asociados. 2. **Campañas de recaudación con barras de progreso en tiempo real:** Visualización interactiva del monto financiero acumulado frente a la meta objetiva. 3. **Checkout de donaciones mediante transferencia bancaria:** Declaración formal de la transferencia con carga de comprobantes y validación administrativa. 4. **Panel de control del administrador:** Permite a los operadores de la cooperadora validar o rechazar transferencias, administrar campañas y gestionar novedades. 5. **Mecanismo de Data Mashup y persistencia híbrida:** Uso de PostgreSQL para la consistencia transaccional ACID y MongoDB para el contenido flexible y multimedia de noticias y detalles enriquecidos. 6. **Automatización de agradecimientos por email (SMTP):** Notificación automática tras la aprobación administrativa de la donación.

1.2 Justificación

- **Necesidad que busca resolver:** Centralizar, formalizar y despapelar la administración de la cooperadora, permitiendo captar aportes de la ciudadanía digitalizada en cualquier momento del día de manera auditable y segura.
- **Aporte para la institución:** Proporciona un canal institucional oficial que jerarquiza la imagen de la cooperadora, reduce significativamente el trabajo administrativo manual de los voluntarios y maximiza la velocidad de recaudación de fondos mediante la visibilidad masiva en internet.

- **Beneficios para los usuarios:** Facilita la colaboración económica rápida sin necesidad de trasladarse físicamente. Otorga transparencia absoluta: el donante puede verificar que su dinero sumó a la barra de progreso de la campaña que eligió y recibe un correo electrónico formal de agradecimiento institucional emitido directamente por el sistema tras la auditoría administrativa.

2. Misión y Visión

2.1 Misión

Proporcionar un portal digital transparente, seguro y eficiente que conecte a la comunidad de Necochea y Quequén con la Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra", facilitando la recaudación de fondos, la gestión de socios y la rendición de cuentas para potenciar el equipamiento y la infraestructura del hospital público.

2.2 Visión

Ser el canal digital líder y referente de transparencia en el ámbito de la salud pública local, logrando la digitalización del 100% de los asociados y aportantes de la Cooperadora, y consolidando la confianza comunitaria a través de la visibilización en tiempo real del impacto directo de sus contribuciones en el hospital.

3. Objetivos del Proyecto

3.1 Objetivo General

Diseñar, desarrollar e implementar una plataforma web con persistencia híbrida y panel administrativo seguro para la Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra" que optimice la captación de socios, la gestión de campañas financieras y la auditoría interna de transferencias bancarias durante el primer semestre de 2026.

3.2 Objetivos Específicos

1. **Arquitectura Híbrida y Mashup:** Diseñar e implementar una base de datos híbrida utilizando PostgreSQL para los datos críticos transaccionales (socios, usuarios, finanzas) y MongoDB para datos no estructurados multimedia (noticias, narrativas enriquecidas), reduciendo la latencia de carga mediante agregación síncrona en el servidor. 2. **Registro de Socios Auditable:** Desarrollar un módulo de registro para asociados que valide campos obligatorios (DNI e email únicos) y permita a los administradores exportar y mantener actualizado el Libro Registro de Asociados digitalizado. 3. **Flujo Contable de Transferencias:** Implementar un flujo de declaración y validación manual de transferencias bancarias en el panel de administración, vinculando las aprobaciones a la actualización automática de las barras de progreso de recaudación. 4. **Seguridad y Tasa de Peticiones:** Garantizar la seguridad de la plataforma

aplicando autenticación JWT, encriptación bcrypt para contraseñas, middleware de sanitización contra inyección NoSQL, protección contra XSS en noticias y rate limiting por IP en endpoints de autenticación y donación. 5. **Notificaciones Automatizadas:** Integrar un servicio de correos SMTP que despache automáticamente correos personalizados de agradecimiento a los donantes una vez que sus transferencias sean aprobadas por el operador. 6. **Validación y Pruebas Automatizadas:** Desarrollar una suite de al menos 45 pruebas de integración automatizadas utilizando Vitest y Supertest que certifique la robustez de las API y el cumplimiento estricto de las reglas de negocio (por ejemplo, impedir aprobaciones que superen la meta restante de la campaña).

4. Identificación de Stakeholders

Stakeholder (Rol)	Interés respecto al proyecto	Nivel de Influencia
---	---	---
Comisión Directiva de la Cooperadora (Product Owner / Cliente)	Maximizar la recaudación de fondos, digitalizar el registro de asociados, y brindar transparencia contable a la comunidad.	Alto
Vecinos y Donantes de Necochea/Quequén (Usuarios Finales)	Donar de forma sencilla mediante transferencia, completar su registro de socios y verificar el impacto real de su aporte.	Medio
Operadores y Personal Administrativo (Usuarios Operacionales)	Disponer de un panel intuitivo para validar donaciones, cargar noticias sin conocimientos técnicos y actualizar campañas.	Medio
Dirección del Hospital Dr. Emilio Ferreyra (Beneficiario Indirecto)	Canalización ágil de recursos económicos de la cooperadora hacia insumos y mejoras de infraestructura edilicia.	Alto
Equipo de Desarrollo (Aramis, Kevin, Thiago, Santiago)	Garantizar la calidad técnica de la plataforma, implementar requerimientos complejos y aprobar el TFI de la cursada.	Alto
Fisco y Órganos de Control (Entes Reguladores)	Cumplimiento de normativas impositivas y de libros legales en la digitalización del registro de asociados.	Medio
Proveedor del Servidor de Correo SMTP (Soporte Técnico)	Garantizar la entrega exitosa de correos electrónicos transaccionales de confirmación.	Bajo

5. Alcance del Proyecto

5.1 Funcionalidades Incluidas

- **Módulo de Autenticación y Cuentas:** Registro de usuarios en rol **socio** e inicio de sesión seguro firmando tokens JWT. Restricción del rol **admin** a cuentas insertadas directamente por base de datos para prevenir auto-escalamiento.
- **Registro en el Libro de Asociados:** Panel del socio para registrar sus datos personales (DNI obligatorio, nombre, dirección) con validación estricta en base de datos PostgreSQL.
- **Visualización de la Campaña del Mes:** Sección destacada en la página de inicio que consume de manera síncrona los montos de PostgreSQL y las narrativas visuales de MongoDB (Data Mashup).
- **Checkout de Declaración de Transferencia:** Formulario interactivo donde el socio declara la transferencia realizada detallando el monto, la fecha, y simula la carga del comprobante bancario.
- **Panel de Administración Integral:** Panel interactivo para el operador de la cooperadora con:
 - Métricas agregadas de recaudación (barra de progreso financiero, total recaudado y socios activos).
 - Listado interactivo de transferencias pendientes de validación con botones de "Aprobar" y "Rechazar" que previenen doble clic.
 - CRUD de campañas de recaudación y edición de noticias de actualidad.
- **Gestor de Noticias y Sanitización:** Publicador dinámico de novedades en MongoDB. Renderizado de artículos con DOMPurify en el frontend para evitar ataques XSS.
- **Protección ante Concurrencia en Donaciones:** Bloqueo exclusivo a nivel de fila (**SELECT ... FOR UPDATE**) en PostgreSQL al validar donaciones concurrentes para evitar colisiones contables.
- **Límite de Recaudación en Campañas:** Bloqueo de aprobaciones de transferencias que superen el saldo restante requerido para completar la campaña de recaudación.
- **Envío de Agradecimiento SMTP:** Conexión asíncrona no bloqueante con Nodemailer para despachar un correo HTML estilizado al socio donante una vez aprobada su donación.
- **Middleware de Seguridad Global:** Rate limiter por IP (100 peticiones globales / 15 min, 10 logins / 15 min, 5 donaciones / hora) y desinfección de consultas contra inyecciones NoSQL (**express-mongo-sanitize**).
- **Suite de Pruebas Automatizadas:** 47 pruebas de integración ejecutadas en un entorno de base de datos de test aislado que valida endpoints CRUD, autenticación, y control de desbordamiento en campañas.

5.2 Funcionalidades Excluidas

- **Procesamiento de Tarjetas de Crédito Reales:** Excluido del proyecto de forma expresa debido a los costos asociados por comisiones de pasarelas de pago y a la necesidad de concentrar la contabilidad en transferencias directas al CBU institucional sin intermediarios comerciales.
- **Sincronización Automática con la API del Banco:** No se realiza la comunicación en tiempo real con el Homebanking de la institución para auto-conciliar transferencias debido a las restricciones de seguridad y el costo elevado de las interfaces bancarias en el mercado nacional.
- **Chat interactivo en tiempo real:** Soporte técnico y consultas directas quedan fuera del alcance del portal web (se manejan vía canales tradicionales de contacto ya existentes).

- **Aplicación Móvil Nativa:** El desarrollo de aplicaciones móviles dedicadas para Android o iOS se excluye del alcance; se garantiza, en cambio, la responsividad absoluta de la aplicación web en dispositivos móviles.

6. Restricciones del Proyecto

- **Restricción de Tiempo:** Plazo de entrega inflexible de 8 semanas, finalizando con la defensa del proyecto integrador el 22 de junio de 2026.
- **Restricción de Recursos Humanos:** Equipo de desarrollo pequeño de 4 integrantes que debe balancear la carga de trabajo de este proyecto con otras asignaturas de la carrera y responsabilidades laborales.
- **Restricción Tecnológica (Académica):** Obligatoriedad de utilizar una **arquitectura híbrida de persistencia** (un motor relacional y un motor documental de manera simultánea) establecida por la cátedra de Programación IV.
- **Restricción de Conocimientos:** Falta de experiencia previa del equipo en la sincronización de bases de datos heterogéneas, la gestión de bloqueos de concurrencia a nivel de base de datos y la orquestación de frameworks de prueba avanzados como Vitest aplicados a bases de datos en paralelo.
- **Restricción de Infraestructura:** Cero presupuesto financiero asignado para hosting y almacenamiento en la nube de producción. Toda la suite debe funcionar de manera óptima localmente y en entornos virtualizados ligeros mediante Docker.

7. Modelo de Desarrollo

Para la gestión de este proyecto, se seleccionó el **Modelo Scrum combinado con un enfoque de Ciclo de Vida Incremental**.

Justificación de la Elección

1. **Entrega de Incrementos Funcionales:** Al tratarse de una cursada estructurada en etapas evaluativas, el desarrollo incremental nos permitió entregar primero un análisis de requerimientos (Etapa 1), luego los wireframes de navegación estática (Etapa 2), posteriormente el backend y diseño de datos (Etapa 3), y finalmente la API conectada, la interfaz responsiva clínica y la suite de pruebas (Etapa 4). 2. **Adaptabilidad ante Cambios:** A lo largo de la cursada surgieron nuevas necesidades, como la de prescindir del flujo de tarjeta de crédito para simplificar el proceso contable en transferencias directas auditables, y la introducción de controles transaccionales de concurrencia. La flexibilidad de los Sprints de Scrum (duración de 2 semanas) facilitó el replanificar y adaptar el backlog de manera ágil sin interrumpir el desarrollo global. 3. **Roles Claros y Mitigación de Cuellos de Botella:** La división formal de responsabilidades en el equipo ágil permitió que cada integrante se focalizara en su área de especialidad (Backend, Frontend, UI/UX, Seguridad/QA) facilitando la integración rápida en la rama **develop** mediante Pull Requests controlados.

8. Aplicación de Scrum

8.1 Roles

- **Product Owner (PO):** Representado por el docente y el contacto institucional de la Cooperadora. Se encarga de validar que las funcionalidades cumplan con las necesidades administrativas y académicas de la asignatura.
- **Scrum Master: Aramis Prieto.** Responsable de facilitar las reuniones de planificación y sincronización, controlar la correcta integración de código en el repositorio Git (**develop** y **main**), y remover impedimentos técnicos vinculados a la base de datos híbrida.
- **Equipo de Desarrollo (Development Team):**
 - **Kevin Nielsen:** Desarrollo del Backend, middleware de rate limiting, validaciones robustas con regex y validator, pasarela de auditoría de transferencias bancarias y servicio de correos SMTP.
 - **Thiago Masson:** Configuración del entorno de dependencias monorepo (migración a pnpm), integración del branding y logotipo oficial de la cooperadora, desarrollo del módulo documental de noticias e implementación de sanitización DOMPurify.
 - **Santiago Ialungo:** Diseño estético clínico de UI/UX, optimización de la paleta de colores y el fondo ECG, desarrollo del Navbar Scroll-Spy, integración de Lenis scroll y la maquetación responsiva del panel de administración.
 - **Aramis Prieto:** Estructuración de la base de datos PostgreSQL y MongoDB, orquestación del Data Mashup, control transaccional de concurrencia mediante **SELECT FOR UPDATE** y programación de la suite de 47 tests automatizados en Vitest.

8.2 Product Backlog Inicial

Código	Item del Backlog	Prioridad	Estimación (SP)	Estado Actual
---	---	---	---	---
PB-01	Configuración de entorno pnpm y arquitectura híbrida Postgres/Mongo	Alta	5	Finalizado
PB-02	Sistema de Registro, Login y seguridad basada en JWT	Alta	5	Finalizado
PB-03	Módulo de registro en el Libro de Asociados (DNI/Domicilio)	Alta	3	Finalizado
PB-04	Creación y listado de Campañas de Recaudación (SQL/NoSQL)	Alta	8	Finalizado
PB-05	Checkout interactivo de declaración de donación por transferencia	Alta	5	Finalizado

PB-06	Panel de administración para auditoría manual de transferencias	Alta	8	Finalizado
PB-07	Módulo dinámico de noticias con protección contra XSS (DOMPurify)	Media	5	Finalizado
PB-08	Rediseño estético clínico del portal e integración de Scroll-Spy/Lenis	Media	8	Finalizado
PB-09	Validaciones robustas, control de inputs y rate limiters por IP	Alta	5	Finalizado
PB-10	Servicio de correos automatizados SMTP para agradecimientos	Media	5	Finalizado
PB-11	Suite de pruebas automatizadas de integración y seguridad (Vitest)	Alta	8	Finalizado

9. Historias de Usuario

HU-01: Registro de Usuarios

- **Como** visitante del portal web,
- **quiero** registrarme ingresando un email y contraseña válidos,
- **para** poder acceder a las secciones privadas y declarar donaciones a mi nombre.
- **Criterios de Aceptación:**

1. El sistema debe validar que el email no esté registrado previamente. 2. La contraseña debe ser hasheada utilizando bcrypt antes de guardarse en PostgreSQL. 3. Al registrarse, el rol asignado por defecto debe ser obligatoriamente **socio**.

HU-02: Alta de Socio en Libro Registro

- **Como** socio registrado,
- **quiero** completar mis datos de perfil (DNI, Nombre, Apellido, Dirección),
- **para** formalizar mi incorporación al Libro Registro de Asociados de la cooperadora.
- **Criterios de Aceptación:**

1. El DNI debe ser numérico y único en la base relacional. 2. Todos los campos obligatorios deben estar validados en el frontend y en el backend. 3. El estado del perfil se inicializa en "Pendiente de Aprobación".

HU-03: Visualización de Campaña Destacada

- **Como** donante potencial,

- **quiero** visualizar la Campaña del Mes en la página de inicio con su meta financiera y barra de progreso,
- **para** comprender la urgencia de la recaudación y decidir en qué colaborar.

- **Criterios de Aceptación:**

1. La barra de progreso debe calcular el porcentaje acumulado en tiempo real según las donaciones aprobadas. 2. Se debe renderizar un esqueleto de carga (skeleton) mientras se realiza el Data Mashup desde el backend. 3. Si no hay campañas activas, se debe mostrar un mensaje alternativo descriptivo.

HU-04: Declaración de Transferencia

- **Como** socio autenticado,
- **quiero** declarar los datos de una transferencia bancaria que realicé a la cuenta de la cooperadora,
- **para** que los administradores puedan auditarla y asignarla a la campaña seleccionada.

- **Criterios de Aceptación:**

1. El socio debe poder seleccionar la campaña de destino, ingresar el monto transferido y cargar una imagen o captura simulada del comprobante. 2. El monto declarado debe ser un valor positivo válido (mayor a \$0). 3. La declaración se registra en PostgreSQL en estado "Pendiente".

HU-05: Validación Administrativa de Donaciones

- **Como** operador administrador de la cooperadora,
- **quiero** ver un listado de transferencias declaradas pendientes de revisión,
- **para** poder aprobarlas o rechazarlas tras contrastarlas con el extracto bancario.

- **Criterios de Aceptación:**

1. Al presionar "Aprobar", el estado de la transferencia cambia a "Aprobada", y el monto de la campaña correspondiente en SQL se incrementa automáticamente. 2. Para evitar clics múltiples accidentales en conexiones lentas, el botón de acción correspondiente debe deshabilitarse dinámicamente de forma inmediata tras el primer clic. 3. Al presionar "Rechazar", el estado cambia a "Rechazada" y no se modifica el acumulado de la campaña.

HU-06: Notificación de Agradecimiento por Correo

- **Como** donante que colaboró con una campaña,
- **quiero** recibir un correo de agradecimiento institucional apenas el administrador valide mi transferencia,
- **para** poseer un comprobante formal y tener la certeza de que mi dinero ingresó a la cuenta.

- **Criterios de Aceptación:**

1. El correo electrónico debe despacharse de forma asíncrona sin bloquear la respuesta de la API de aprobación. 2. El correo debe contener detalles estilizados en formato HTML con el logotipo oficial, nombre del donante, monto de la donación, y nombre de la campaña.

HU-07: Límite de Recaudación en Campañas

- **Como** operador de la cooperadora,
- **no quiero** que se aprueben transferencias que superen el saldo restante de la meta de una campaña,
- **para** evitar recaudar de más sobre un objetivo financiero que ya se cumplió.
- **Criterios de Aceptación:**

1. Si una transferencia pendiente tiene un monto superior al saldo necesario para completar la campaña, el backend debe rechazar la aprobación devolviendo un código de error explícito. 2. Se debe bloquear la declaración de nuevas donaciones en el frontend para campañas que ya hayan alcanzado el 100% de su meta.

HU-08: Gestión de Noticias de Actualidad

- **Como** operador administrador,
- **quiero** redactar y publicar noticias de actualidad con galerías fotográficas en el portal,
- **para** mantener informada a la comunidad sobre las compras realizadas y eventos benéficos.
- **Criterios de Aceptación:**

1. Los datos multimedia y el texto flexible se guardan en la colección MongoDB. 2. El frontend debe sanitizar el cuerpo HTML de las noticias usando DOMPurify para neutralizar cualquier inyección de scripts maliciosos (XSS).

HU-09: Control de Tasa (Rate Limiting)

- **Como** administrador de la infraestructura del portal,
- **quiero** limitar la cantidad de peticiones consecutivas permitidas por dirección IP,
- **para** mitigar ataques de fuerza bruta en el login y saturación maliciosa en el servidor.
- **Criterios de Aceptación:**

1. El endpoint de autenticación **/api/auth/login** debe limitar a un máximo de 10 intentos fallidos por IP cada 15 minutos. 2. Las API generales de donación deben limitar a 5 declaraciones por hora por dirección IP. 3. Si se supera el límite establecido, la API debe responder con el código HTTP 429 "Too Many Requests".

HU-10: Suite de Pruebas de Reglas de Negocio

- **Como** desarrollador del sistema,
- **quiero** contar con una suite de pruebas automatizadas sobre una base de datos aislada,
- **para** asegurar que las refactorizaciones o futuros cambios no rompan los flujos transaccionales críticos.

- **Criterios de Aceptación:**

1. Las pruebas deben ejecutarse en bases de datos de test (**cooperadora_db_test** y **cooperadora_nosql_test**) sin alterar los datos de desarrollo ni de producción. 2. La suite debe correr de manera secuencial y limpiar los registros insertados de forma automática entre cada test.

10. Requerimientos

10.1 Requerimientos Funcionales (RF)

1. **RF-01 (Autenticación):** El sistema debe permitir a los usuarios registrarse e iniciar sesión de forma segura generando y validando tokens JWT. 2. **RF-02 (Gestión de Roles):** El sistema debe restringir el acceso a los métodos de escritura (POST, PUT, DELETE) de campañas y noticias únicamente a usuarios con el rol **admin**. 3. **RF-03 (Registro de Datos de Socio):** La plataforma debe permitir a los usuarios en rol **socio** completar la información obligatoria (DNI, domicilio, teléfono) para ser incluidos en el Libro de Asociados. 4. **RF-04 (Mashup de Campañas):** El backend debe fusionar los datos financieros de SQL y la narrativa multimedia de NoSQL al consultar el detalle de una campaña, retornando un único objeto JSON estructurado. 5. **RF-05 (Declaración de Donación):** El sistema debe permitir a los socios logueados declarar una transferencia realizada mediante un formulario, registrando el monto y asociando el comprobante. 6. **RF-06 (Auditoría Administrativa):** El panel de administración debe proveer un listado de transferencias declaradas y permitir al operador marcarlas como "Aprobada" o "Rechazada". 7. **RF-07 (Límite de Campaña):** La API de donaciones debe rechazar cualquier transacción que exceda el remanente financiero necesario para alcanzar el 100% de la meta de la campaña. 8. **RF-08 (Gestión Documental de Noticias):** El sistema debe permitir el almacenamiento de artículos de noticias multimedia con formato libre en MongoDB. 9. **RF-09 (Envío SMTP):** El backend debe enviar un correo electrónico HTML formal de agradecimiento al email del socio inmediatamente tras confirmarse la validez de su donación. 10. **RF-10 (Control de Transacciones Concurrentes):** El backend debe emplear bloqueos exclusivos de fila (**SELECT FOR UPDATE**) al actualizar los saldos de campañas en SQL para evitar inconsistencias en el acumulado de recaudaciones simultáneas.

10.2 Requerimientos No Funcionales (RNF)

1. **RNF-01 (Seguridad - Hashing):** Todas las contraseñas de los usuarios deben almacenarse de forma irreversible utilizando el algoritmo de hashing **bcryptjs** con un factor de costo mínimo de 10. 2. **RNF-02 (Seguridad - Inyecciones):** El backend debe neutralizar inyecciones de código NoSQL sanitizando caracteres reservados en las solicitudes JSON usando **express-mongo-sanitize**. 3. **RNF-03 (Seguridad - XSS):** El frontend debe sanitizar dinámicamente todo el contenido HTML insertado por administradores antes de renderizarlo, empleando **DOMPurify** para neutralizar scripts maliciosos. 4. **RNF-04 (Rendimiento - Latencia de Mashup):** Las consultas de agregación de datos híbridos (Data Mashup) deben ejecutarse en paralelo usando **Promise.all** para asegurar que el tiempo de respuesta del backend sea inferior a 150 ms bajo condiciones normales de red. 5. **RNF-05 (Disponibilidad y Tasa):** El sistema debe resistir ataques básicos de denegación de servicio (DoS) bloqueando mediante HTTP 429 a cualquier IP que supere las 100 peticiones globales por cada ventana de 15 minutos.

11. Priorización de Requerimientos

La priorización se realizó bajo la metodología MoSCoW, enfocada en asegurar un Producto Mínimo Viable (MVP) completamente funcional antes del cierre del ciclo académico.

- **Prioridad ALTA (Imprescindibles para el core de negocio):**

- **RF-01, RF-02, RF-03, RF-05, RF-06, RF-10 y RNF-01, RNF-02.**

- *Justificación:* Sin autenticación segura, validación de transferencias y control transaccional de concurrencia, el sistema carece de valor administrativo y expone la integridad contable de la cooperadora (riesgo de inconsistencia en saldos de dinero).

- **Prioridad MEDIA (Aportan alto valor y mejoran la experiencia institucional):**

- **RF-04, RF-07, RF-09 y RNF-03, RNF-04, RNF-05.**

- *Justificación:* El control de límites de campaña, el Data Mashup optimizado y el envío de agradecimientos SMTP elevan la calidad del software y mejoran significativamente el engagement y confianza del donante, pero el negocio básico puede operar manualmente si no estuvieran de inmediato.

- **Prioridad BAJA (Deseables a futuro):**

- **RF-08.**

- *Justificación:* El gestor de noticias dinámico es sumamente útil para la comunicación, pero no afecta el flujo core transaccional de recaudación ni la gestión de socios del Libro de Asociados.

12. Gestión de Riesgos

- **R-01: Inconsistencia Contable por Concurrencia (Técnico):** Dos donaciones concurrentes aprobadas en el mismo instante sobrescriben mutuamente el **monto_actual** de la campaña, perdiéndose la sumatoria de una de ellas.
- **R-02: Escalabilidad Maliciosa de Roles (Técnico):** Un atacante intercepta o manipula la API de registro público para auto-asignarse el rol **admin** y modificar las metas financieras a su favor.
- **R-03: Ataque XSS a través de Noticias (Técnico):** Un operador comprometido o un atacante inserta código JavaScript malicioso en los textos enriquecidos de MongoDB, ejecutando código arbitrario en las sesiones de los socios visitantes.
- **R-04: Retraso en la Entrega Final (Gestión):** Sobrecarga de tareas universitarias y laborales de los integrantes del equipo que impide culminar la integración a tiempo para la fecha de defensa (22 de junio).
- **R-05: Falta de Infraestructura y Servidor SMTP Real (Gestión):** No disponer de credenciales reales de correo o de un servidor de correo corporativo para despachar las notificaciones transaccionales.
- **R-06: Inyecciones NoSQL en MongoDB (Técnico):** Inyección de operadores de consulta MongoDB (ej. **{"\$gt": ""}**) en solicitudes de login para eludir la autenticación sin conocer la contraseña.

- **R-07: Resistencia al Cambio de los Operadores (Humano):** El personal administrativo tradicional de la Cooperadora prefiere seguir usando planillas de papel y rechaza usar el panel administrativo del portal.
- **R-08: Saturación del Servidor por Fuerza Bruta (Técnico):** Ataque automatizado de fuerza bruta sobre el login que agota los recursos de memoria y CPU del servidor local.
- **R-09: Regresión de Software por Integración Rápida (Gestión):** Romper componentes existentes (como la persistencia NoSQL de noticias) al integrar la lógica de transferencias en PostgreSQL.
- **R-10: Incumplimiento de Metas por Donaciones Huérfanas (Negocio):** Socios que declaran transferencias falsas o montos erróneos inflando artificialmente el progreso de la campaña sin que el dinero real haya ingresado.

13. Matriz de Riesgos

Id	Descripción	Impacto	Probabilidad	Nivel
R-01	Inconsistencia Contable por Concurrencia	Media	Alto	Alto
R-02	Escalabilidad Maliciosa de Roles	Baja	Alto	Medio
R-03	Ataque XSS a través de Noticias	Media	Medio	Medio
R-04	Retraso en la Entrega Final	Alta	Alto	Alto
R-05	Falta de Infraestructura y SMTP Real	Alta	Medio	Medio
R-06	Inyecciones NoSQL en MongoDB	Baja	Alto	Medio
R-07	Resistencia al Cambio de los Operadores	Media	Medio	Medio
R-08	Saturación del Servidor por Fuerza Bruta	Media	Medio	Medio
R-09	Regresión de Software por Integración	Media	Medio	Medio
R-10	Incumplimiento por Donaciones Huérfanas	Alta	Alto	Alto

14. Estrategias de Mitigación

- **Mitigación R-01 (Concurrencia):** Implementación de transacciones en Sequelize con bloqueo explícito de fila (**SELECT ... FOR UPDATE** mediante **lock: transaction.LOCK.UPDATE**). Las solicitudes simultáneas se encolan automáticamente esperando la liberación del lock.
- **Mitigación R-02 (Roles):** El endpoint **/api/auth/register** tiene harcodeado el rol **socio** en el backend. Las cuentas **admin** solo pueden crearse mediante una sentencia SQL directa ejecutada de manera local por el administrador de base de datos.
- **Mitigación R-03 (XSS):** Filtrado preventivo en el frontend utilizando la biblioteca **DOMPurify** sobre el cuerpo HTML recibido desde MongoDB antes de inyectarlo en la vista con **dangerouslySetInnerHTML**.
- **Mitigación R-04 (Retraso):** División del desarrollo en sprints quincenales e integración constante en la rama **develop**. Seguimiento de tareas prioritarias en **TODO.md** para evitar desviaciones del alcance.
- **Mitigación R-05 (SMTP):** Parametrización de variables de entorno SMTP en el backend. En desarrollo local se utiliza una cuenta de prueba configurada de forma segura o un servicio SMTP simulado (Mailtrap / Gmail con contraseña de aplicación).
- **Mitigación R-06 (Inyecciones NoSQL):** Empleo de la biblioteca middleware **express-mongo-sanitize** para eliminar de forma automática cualquier carácter especial como **\$** o **.** en el cuerpo de las peticiones.
- **Mitigación R-07 (Resistencia):** Diseño de una interfaz administrativa clínica limpia, minimalista y autodescriptiva que no requiera capacitación técnica. Mantener el flujo de validación manual en pocos clics.
- **Mitigación R-08 (Fuerza Bruta):** Implementación de rate limiters basados en la dirección IP mediante **express-rate-limit**, bloqueando temporalmente a clientes sospechosos.
- **Mitigación R-09 (Regresión):** Construcción de una robusta suite de pruebas con Vitest que valida de manera automática que todos los endpoints respondan correctamente ante refactorizaciones de código.
- **Mitigación R-10 (Donaciones Huérfanas):** Mantener el estado "Pendiente" para las donaciones declaradas. La barra de progreso de la campaña solo se incrementa una vez que el administrador valida que el dinero impactó efectivamente en el extracto bancario real de la institución.

15. Calidad del Software

Considerando las características del estándar de calidad analizados, el proyecto garantiza cada aspecto de la siguiente manera:

1. **Adecuación Funcional:** La plataforma cubre con exactitud el ciclo de negocio real: registro de socios en el libro legal, visualización fidedigna de campañas, declaración y validación de transferencias, y comunicación de novedades. 2. **Usabilidad:** Rediseño UI/UX adaptado a la salud, con una interfaz clara basada en colores pasteles, una cuadrícula ECG de fondo que contextualiza el portal, animaciones sutiles, estados de carga (skeletons) y scroll inercial suave. El Navbar destaca la sección activa mediante Scroll-Spy. 3. **Eficiencia:** Agregación síncrona en base de datos híbrida (Data Mashup) utilizando **Promise.all** para ejecutar de manera paralela las consultas relacionales y documentales, evitando bloqueos de red secuenciales. 4. **Fiabilidad:** Prevención de colisiones de datos bajo concurrencia concurrente en SQL mediante bloqueo exclusivo de fila y manejo transaccional a nivel de motor de datos. 5. **Seguridad:** Uso de tokens JWT firmados digitalmente para la autorización, almacenamiento seguro de contraseñas mediante hash **bcryptjs**, desinfección de inputs para evitar inyecciones SQL/NoSQL y sanitización contra inyecciones XSS. 6. **Mantenibilidad:** Estructuración de código modular (controladores, modelos y rutas diferenciados), suite de testing automatizada para prevención de regresiones y control estricto de dependencias en un monorepo coordinado con **pnpm**.

16. Estrategia de Testing

El plan de testing se compone de tres etapas estratégicas:

1. Pruebas de Integración Automatizadas:

- **Herramienta:** Vitest y Supertest.
- **Objetivo:** Validar las respuestas HTTP, el enrutamiento y las reglas de negocio críticas de la API (alta de socios, registro, Mashup y flujos de donaciones).
- **Implementación:** 47 casos de prueba ejecutados sobre bases de datos locales dedicadas a test (**cooperadora_db_test** y **cooperadora_nosql_test**) con limpieza automática mediante scripts de setup.

2. Pruebas de Concurrencia:

- **Herramienta:** Scripts personalizados simulando múltiples llamadas simultáneas de aprobación al endpoint de donaciones.
- **Objetivo:** Asegurar que el bloqueo exclusivo **SELECT FOR UPDATE** en PostgreSQL encole las peticiones de forma ordenada y no existan condiciones de carrera.

3. Pruebas de Seguridad (Penetración Básica):

- **Objetivo:** Verificar la inmutabilidad de los limitadores de tasa (Rate Limiting) y comprobar que el middleware bloquea inyecciones de operadores NoSQL y payloads maliciosos de JavaScript en la carga de noticias.

17. Deuda Técnica

La deuda técnica en el desarrollo del portal web se identifica en tres aspectos principales:

- **Origen:**

- **Urgencia del Cronograma:** La necesidad de completar el flujo de donaciones bancarias a tiempo para las pre-entregas forzó al equipo a delegar la validación e integración del scroll a etapas posteriores (Lenis scroll presentaba conflictos con estilos CSS globales).
- **Sincronización Híbrida Manual:** Ante la falta de un middleware ORM híbrido nativo, la sincronización entre el ID de campaña SQL y el detalle de campaña en MongoDB se realiza de manera manual en la capa de negocio del controlador backend.

- **Consecuencias:**

- Incremento del esfuerzo requerido para añadir nuevas vistas multimedia.
- Fragilidad en el acople de la maquetación de frontend si el archivo global CSS crece desordenadamente.

- **Acciones Correctivas:**

- Creación de la suite de Vitest de forma temprana para detectar regresiones ante cambios en las bases de datos.
- Refactorización del Lenis scroll migrando formalmente a **@lenis/react**, limpiando los remanentes obsoletos en **index.css** para estabilizar el renderizado del cliente.

PARTE 2: Estimación, Costos, Planificación y Gestión de Cambios

19. Presentación del Proyecto (Estimación)

19.1 Estrategia de Estimación Seleccionada

Para el desarrollo de la segunda etapa del proyecto se combinaron tres metodologías complementarias: 1. **Work Breakdown Structure (WBS)**: Descomposición jerárquica del proyecto en componentes menores para identificar con precisión todas las tareas de diseño, codificación, aseguramiento de calidad y despliegue. 2. **Juicio de Expertos**: Basado en la experiencia del equipo en proyectos académicos anteriores de Programación y Base de Datos para establecer una línea base de complejidad. 3. **Story Points mediante Planning Poker (Estimación Ágil)**: Asignación de puntos de historia para estimar el esfuerzo y complejidad relativa de cada requerimiento del Backlog, utilizando la escala modificada de Fibonacci (1, 2, 3, 5, 8, 13).

Justificación de la Elección

La combinación de Story Points y WBS resulta óptima para el modelo Scrum. Estimar en horas directas en un equipo de estudiantes suele ser inexacto debido a la disparidad de horarios y curvas de aprendizaje individuales. Los **Story Points** abstraen el factor tiempo y se concentran en la complejidad técnica y el riesgo. Por su parte, la **WBS** garantiza que no se omitan actividades fundamentales no funcionales (como la configuración del monorepo, las migraciones de base de datos o la configuración del servidor SMTP).

19.2 Descomposición de Funcionalidades (WBS)

El proyecto se descompuso en las siguientes tareas dentro de la estructura de WBS:

Nivel WBS	ID Tarea	Nombre de la Tarea / Componente	Descripción
---	---	---	---
1.0	INF	Infraestructura y Monorepo	Inicialización técnica del entorno de desarrollo
1.1	INF-01	Configuración de pnpm workspace	Migración del esquema npm tradicional a pnpm monorepo.
1.2	INF-02	Dockerización de bases de datos	Configuración de contenedores locales para Postgres y MongoDB.
2.0	AUT	Módulo de Autenticación	Seguridad y perfiles de acceso
2.1	AUT-01	Login/Registro Backend JWT	Desarrollo de endpoints en backend, firma y verificación de tokens.
2.2	AUT-02	Formulario de Registro en Cliente	Interfaz gráfica responsiva y redirección inteligente tras login.

3.0	SOC	Módulo de Gestión de Socios	Administración del Libro de Asociados
3.1	SOC-01	API de Perfil de Socio	CRUD relacional en Postgres para completar DNI y domicilio.
3.2	SOC-02	Interfaz de Autogestión del Socio	Panel privado donde el socio visualiza y edita su información.
4.0	CAM	Módulo de Campañas	Visualización financiera y narrativa híbrida
4.1	CAM-01	Modelos SQL/NoSQL de Campañas	Creación de tablas de metas (Postgres) y documentos multimedia (Mongo).
4.2	CAM-02	Endpoint de Data Mashup	Agregación síncrona en backend con Promise.all para campañas.
4.3	CAM-03	Componentes de visualización y progreso	Barra de progreso e indicadores interactivos en la Home del cliente.
5.0	DON	Módulo de Donaciones	Declaración de aportes por transferencia
5.1	DON-01	API de declaración de transferencias	Rutas y lógica para almacenar donaciones pendientes con comprobante.
5.2	DON-02	Control transaccional de concurrencia	Bloqueo SELECT FOR UPDATE para evitar colisión de montos.
5.3	DON-03	Control de límites financieros	Middleware que restringe aprobaciones que excedan la meta restante.
6.0	ADM	Panel de Administración	Auditoría contable y gestión de contenidos
6.1	ADM-01	Interfaz de control administrativo	Listado de transferencias y botones de acción rápida sin doble clic.
6.2	ADM-02	CRUD de campañas y noticias	Formularios para que el operador administre la plataforma.
7.0	SEG	Seguridad y Calidad	Robustez e integridad del portal
7.1	SEG-01	Middleware de Rate Limiting e Inyecciones	Configuración de limitadores por IP y sanitización en Express.
7.2	SEG-02	Servicio de correos SMTP	Nodemailer asíncrono para correos de agradecimiento.
7.3	SEG-03	Suite de pruebas de integración	47 pruebas automatizadas en Vitest con entornos de test aislados.

19.3 Estimación de Esfuerzo

Se estimó el esfuerzo relativo de cada tarea utilizando Story Points (SP) justificados según complejidad:

ID Tarea	Estimación (SP)	Justificación de los Story Points
---	:-:	---

INF-01	2	Tarea sencilla pero crítica de estandarización de dependencias monorrepo.
INF-02	3	Configuración y prueba de conectividad simultánea de los dos motores locales en Docker.
AUT-01	5	Complejidad de seguridad: firma de JWT, contraseñas bcrypt y expiración segura de sesión.
AUT-02	3	Desarrollo del formulario de login y redirección inteligente reteniendo parámetros de campaña.
SOC-01	3	Estructuración del modelo relacional en Sequelize con DNI únicos obligatorios.
SOC-02	3	Maquetación responsiva del panel privado del socio para autogestión de datos.
CAM-01	5	Diseño del esquema híbrido de datos cruzando relaciones SQL y documentos NoSQL.
CAM-02	5	Desarrollo del controlador Mashup y optimización con Promise.all para evitar latencia de red.
CAM-03	3	Integración en la interfaz de usuario con barra de progreso, Lenis scroll y skeletons de carga.
DON-01	5	Diseño de la tabla de transferencias y rutas de declaración con adjunto de comprobantes.
DON-02	8	Alta complejidad: gestión transaccional SQL y bloqueos de fila exclusivos para la concurrencia.
DON-03	5	Lógica de validación contable para el saldo de la campaña y rechazos controlados.
ADM-01	5	UI interactiva para aprobar/rechazar transferencias inhabilitando botones para evitar doble envío.
ADM-02	5	Formularios CRUD administrativos, sanitizados contra código malicioso (DOMPurify).
SEG-01	3	Integración de Express-rate-limit y Mongo-sanitize en la pila de middlewares central.
SEG-02	5	Configuración de Nodemailer, plantillas HTML adaptativas y flujo asíncrono no bloqueante.
SEG-03	8	Alta complejidad: diseño de 47 tests en base aislada con borrado automático tras ejecución.
TOTAL	77 SP	Esfuerzo global del proyecto estimado en puntos de historia.

19.4 Análisis de Factores que Afectan la Estimación

- **Complejidad Técnica (Persistencia Híbrida):** La falta de soporte nativo para consultas unificadas SQL/NoSQL requirió codificar lógica adicional en el backend (Mashup), lo que incrementó las estimaciones de las tareas de campañas.
- **Curva de Aprendizaje en Concurrencia:** La lógica de locks en Sequelize (**SELECT FOR UPDATE**) requirió investigación profunda y pruebas exhaustivas, elevando el riesgo y esfuerzo estimado de la tarea **DON-02** al máximo de la escala.

- **Integración del Scroll y CSS:** Los problemas imprevistos en la maquetación CSS global al integrar Lenis scroll generaron un desvío técnico que afectó los tiempos del frontend.
- **Dependencias de SMTP Externo:** La configuración de servidores SMTP reales o cuentas de aplicación de Gmail introdujo dependencias externas de red que incrementaron la variabilidad en las pruebas de notificaciones.

20. Recursos del Proyecto

20.1 Recursos Humanos

Para llevar adelante el desarrollo, los 4 integrantes del grupo asumieron roles técnicos complementarios:

- **Scrum Master & Lead Backend Developer (Aramis Prieto):**
 - *Responsabilidades:* Orquestación de bases de datos híbridas, transaccionalidad contable y concurrencia (locks SQL), control de integración Git y suite de tests en Vitest.
 - *Participación:* 30%.
- **Security & Backend Developer (Kevin Nielsen):**
 - *Responsabilidades:* Implementación de JWT, desarrollo de APIs de transferencias y perfiles, middlewares de sanitización NoSQL, control de tasa (rate limiting) y servicio de correos SMTP.
 - *Participación:* 30%.
- **UI/UX & Lead Frontend Developer (Santiago Ialungo):**
 - *Responsabilidades:* Diseño visual con estética clínica, responsividad, integración de Lenis scroll y Navbar Scroll-Spy, maquetación de gráficos en panel de administración.
 - *Participación:* 20%.
- **Full Stack Developer & Branding Specialist (Thiago Masson):**
 - *Responsabilidades:* Gestión del monorrepo pnpm, integración de logotipos oficiales, desarrollo del gestor de noticias MongoDB, implementación de sanitización DOMPurify.
 - *Participación:* 20%.

20.2 Recursos Tecnológicos

- **Lenguajes y Entornos de Ejecución:** JavaScript (Node.js v18+, React con Vite).
- **Frameworks y Librerías de Backend:** Express (servidor API), Sequelize (ORM SQL), Mongoose (ODM NoSQL), Nodemailer (SMTP).
- **Frameworks y Librerías de Frontend:** React.js, Tailwind CSS (estilos responsivos), Lenis (scroll inercial suave), [@lenis/react](#), DOMPurify (sanitización XSS).
- **Bases de Datos:** PostgreSQL (transaccional ACID) y MongoDB (documental multimedia).

- **Seguridad:** bcryptjs (hashing de claves), jsonwebtoken (JWT), express-rate-limit (tasa por IP), express-mongo-sanitize (inyecciones).
- **Testing:** Vitest, Supertest.
- **Control de Versiones y Dependencias:** Git, GitHub, pnpm (monorepo workspace).
- **Infraestructura de Desarrollo:** Docker Desktop (para levantar Postgres y Mongo localmente).

20.3 Recursos de Infraestructura

- **Estaciones de Trabajo:** Computadoras personales de desarrollo (arquitectura Apple Silicon / Intel Core i7 con 16GB RAM).
- **Servidores Virtuales de Desarrollo (Simulados):** Contenedores Docker locales para la persistencia SQL y NoSQL aisladas.
- **Servidor de Correo SMTP:** Servidor SMTP de desarrollo (Gmail con contraseña de aplicación o Mailtrap) para el envío y validación de notificaciones de agradecimiento.
- **Red Local / Proxy:** Proxy reverso de Vite configurado en el puerto 3000 apuntando hacia el puerto 5001 del backend para eludir políticas de CORS sin requerir certificados de producción en desarrollo.

21. Costos del Proyecto

21.1 Costos Humanos

Aunque el proyecto se desarrolló ad-honorem en el marco de la cursada universitaria, se estimó un presupuesto profesional real asumiendo tarifas del mercado de desarrollo de software en Argentina para mediados de 2026:

- **Duración total:** 8 semanas (Sprint de desarrollo).
- **Dedicación semanal por desarrollador:** 8 horas promedio (dedicación parcial).
- **Horas totales estimadas por desarrollador:** 64 horas.
- **Horas totales del equipo (4 integrantes):** 256 horas.
- **Valor hora de desarrollo promedio asumido:** \$15 USD (tarifa junior/semi-senior promedio).

Nombre	Rol	Horas	Tarifa	Total
Aramis Prieto	Scrum Master / Lead Backend	64 hs	\$15 USD	\$960 USD
Kevin Nielsen	Security / Backend Developer	64 hs	\$15 USD	\$960 USD
Santiago Ialungo	UI/UX / Lead Frontend Dev	64 hs	\$15 USD	\$960 USD

Thiago Masson	Full Stack / Branding Dev	64 hs	\$15 USD	\$960 USD
TOTAL HUMANO		256 hs		\$3.840 USD

21.2 Costos Tecnológicos

Para la puesta en producción real de la plataforma, se estiman los siguientes costos anuales de licenciamiento e infraestructura en la nube (proveedores como AWS o DigitalOcean y servicios locales):

Concepto Tecnológico	Descripción / Proveedor	Costo Mensual	Costo Anual
---	---	---	---
Hosting VPS	Instancia Linux básica en la nube (Node.js/React)	\$10 USD	\$120 USD
Base de Datos SQL	PostgreSQL administrado (con backups diarios)	\$15 USD	\$180 USD
Base de Datos NoSQL	MongoDB Atlas (Tier compartido escalable)	\$9 USD	\$108 USD
Dominio Web	Dominio .org.ar para instituciones sin fines de lucro	-	\$15 USD
Certificado SSL	Let's Encrypt (Configuración de renovación automática)	Gratis	Gratis
Servicio de Email SMTP	SendGrid / Mailgun (Plan básico de hasta 10.000 envíos)	\$15 USD	\$180 USD
Licencia Docker/Tools	Docker Desktop Personal / VS Code / Git	Gratis	Gratis
TOTAL TECNOLÓGICO		\$49 USD	\$603 USD

21.3 Costos de Operación y Mantenimiento (Post-producción mensual)

Una vez desplegado el sistema, se requieren tareas periódicas para garantizar la disponibilidad y seguridad del portal:

- **Soporte y Actualizaciones de Seguridad:** 4 horas mensuales de mantenimiento técnico (prevención de vulnerabilidades y actualización de dependencias npm/pnpm). Costo estimado: **\$60 USD/mes** (\$720 USD/año).
- **Monitoreo y Respallos:** Control automático de logs de error y backups de base de datos relacional para evitar pérdida de registros de socios. Costo estimado: **\$20 USD/mes** (\$240 USD/año).
- **Total de Operación y Mantenimiento Anual: \$960 USD.**

21.4 Costo Total Estimado Consolidado

Categoría de Costo	Costo Inicial / Desarrollo	Costo Anual Recurrente (Operación)
---	---	---
Recursos Humanos (Desarrollo)	\$3.840 USD	-
Recursos Tecnológicos (Nube)	-	\$603 USD
Operación y Mantenimiento	-	\$960 USD
TOTAL CONSOLIDADO	\$3.840 USD	\$1.563 USD

*Nota: El costo total para el primer año de vida útil del sistema (desarrollo + operación) asciende a **\$5.403 USD**.*

22. Planificación del Proyecto

22.1 Cronograma General

El cronograma del proyecto se estructuró a lo largo del cuatrimestre académico con las siguientes etapas:

Fase / Actividad	Fechas	Duración	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
---	---	---												
Análisis e Investigación (Etapa 1)	01/04 - 15/04	14 días												
Diseño de Wireframes (Etapa 2)	15/04 - 29/04	14 días												
Diseño de Datos y Backend (Etapa 3)	29/04 - 13/05	14 días												
Desarrollo e Integración (Etapa 4)	13/05 - 03/06	21 días												
Testing y Seguridad	03/06 - 13/06	10 días												
Implementación Local y Defensa	13/06 - 22/06	9 días												

22.2 Hitos del Proyecto

1. **Hito 1 (15 de abril de 2026):** Documento de Análisis de Requisitos y Etapa 1 aprobado. Establece el público objetivo y la justificación. 2. **Hito 2 (29 de abril de 2026):** Diseño de Wireframes y flujo de pantallas estáticas (Etapa 2) completado. Aprobación de la estructura del sitio. 3. **Hito 3 (13 de mayo de 2026):** Arquitectura híbrida de base de datos definida y API CRUD de backend estructuradas (Etapa 3). Conexión

exitosa a Postgres y MongoDB. 4. **Hito 4 (3 de junio de 2026): Integración de Frontend con Backend y checkout de transferencias finalizado.** Barras de progreso financiero interconectadas en el panel administrativo. 5. **Hito 5 (12 de junio de 2026): Suite de testing automatizada al 100% y rate limiting implementado.** Plataforma estable y desinfectada contra ataques XSS e inyecciones. 6. **Hito 6 (22 de junio de 2026): Defensa Oral y Cierre del Proyecto.** Presentación final integradora ante la mesa evaluadora.

22.3 Entregables

Entregable	Descripción	Momento de Entrega
---	---	---
Entregable 1: Wireframes HTML	Archivos HTML estáticos con la propuesta de diseño visual (Home, Login, Admin, Buscador).	Fin de la Semana 4
Entregable 2: Código Backend API	Repositorio backend en Node/Express con persistencia híbrida configurada y validada.	Fin de la Semana 6
Entregable 3: Portal Web Integrado	Monorrepo pnpm conteniendo el frontend React conectado y el panel administrativo interactivo.	Fin de la Semana 8
Entregable 4: Suite de Pruebas	Suite de 47 pruebas automatizadas en Vitest integradas y listas para ejecución.	Fin de la Semana 9
Entregable 5: Documento de Gestión	Informe PDF formal unificado de gestión del desarrollo de software (Parte 1 y Parte 2).	Fin de la Semana 10

22.4 Dependencias

- **La codificación del frontend (React) depende de la API estable del Backend:** No se pueden integrar las barras de progreso interactivo sin la respuesta JSON unificada del endpoint de Data Mashup de Campañas.
- **La validación del checkout de donación depende del alta de perfiles de socios:** Para declarar una transferencia, el socio debe haber ingresado previamente su DNI en el panel privado de autogestión (Libro de Asociados).
- **El envío de correos SMTP depende de la aprobación contable:** El despachador de Nodemailer no puede gatillarse de forma huérfana; requiere obligatoriamente que el operador presione "Aprobar" sobre una transferencia en el panel.
- **La finalización y aprobación del proyecto académico depende de la suite de testing:** La cátedra exige que las reglas críticas de desbordamiento de metas financieras y concurrencia estén cubiertas por pruebas automatizadas antes de la defensa oral.

23. Gestión de Cambios

23.1 Escenarios de Cambio

Durante el ciclo de vida del desarrollo se presentaron cinco situaciones de cambio significativas que requirieron la adaptación del plan original:

1. **Cambio 1: Remoción de la Pasarela de Tarjeta de Crédito Simulada:** Inicialmente se planeaba incluir cobros directos por tarjeta de crédito en el frontend. Sin embargo, tras dialogar con la cooperadora, se identificó que las comisiones comerciales y la complejidad impositiva desaconsejaban este canal, prefiriendo concentrar toda la recaudación en transferencias bancarias directas declaradas y auditadas manualmente. 2. **Cambio 2: Conflicto e Incompatibilidad del Lenis Scroll:** Durante el rediseño clínico, la biblioteca Lenis (para desplazamiento suave) generaba bloqueos visuales y colisiones con los selectores de scroll nativos en **index.css**, ralentizando la carga del frontend en navegadores Firefox y móviles. 3. **Cambio 3: Necesidad de Control Contable Transaccional:** Se detectó el riesgo de concurrencia al realizar múltiples pruebas simultáneas de aprobación sobre una misma campaña, forzando a reestructurar la consulta de actualización con un bloqueo de fila (**SELECT FOR UPDATE**). 4. **Cambio 4: Incorporación de Límites de Recaudación:** Originalmente se permitía declarar y aprobar cualquier monto sobre una campaña. Se solicitó añadir una validación contable estricta en caliente que bloquee la aprobación de transferencias que superen el monto restante para la meta de la campaña. 5. **Cambio 5: Sanitización Obligatoria XSS en Noticias:** Conectores HTML ricos inyectados en la colección MongoDB de noticias evidenciaron vulnerabilidades de inyección de scripts (XSS). Se requirió incorporar la biblioteca DOMPurify en el frontend de forma urgente.

23.2 Análisis de Impacto de los Cambios

---	---	---	---	---
Cambio 1: Remoción Tarjeta	Reducción de alcance: Se eliminaron las API de cobro simulado y controladores obsoletos.	Neutro: Liberó horas de desarrollo que se reasignaron a auditoría de transferencias.	Reducción: Evitó el análisis de costos de pasarelas de terceros.	Alta: Focalizó la contabilidad del sistema en un flujo 100% auditable.
Cambio 2: Lenis Scroll	Neutro: No modificó los requerimientos funcionales de la UI.	Desvío de 2 días: Se requirió refactorizar el wrapper CSS y migrar a @lenis/react .	Neutro: Cero costos de licenciamiento (código abierto).	Alta: Garantizó una navegación suave, fluida y profesional en todo tipo de pantallas.
Cambio 3: Concurrencia SQL	Incremento técnico: Incorporación de bloqueos de fila relacionales y control transaccional.	Desvío de 3 días: Requirió escribir tests de concurrencia específicos.	Neutro	Crítica: Elevó la robustez financiera y fiabilidad de los datos al 100% (cero colisiones).

Cambio 4: Límite de Campañas	Incremento de alcance: Nuevas reglas de negocio de validación de saldos en backend.	Desvío de 2 días: Desarrollo de endpoints de control y suite de pruebas en Vitest.	Neutro	Alta: Evitó la sobredonación y garantizó la coherencia contable frente a metas financieras.
Cambio 5: Sanitización XSS	Incremento de seguridad: Sanitización en el renderizado con DOMPurify.	Desvío de 1 día: Incorporación de dependencia y refactorización de vistas.	Neutro	Alta: Clausuró vulnerabilidades críticas de seguridad, protegiendo las sesiones de los socios.

23.3 Estrategias de Gestión de Cambios

Para evitar el descontrol del alcance (*scope creep*) ante estas situaciones de cambio, el equipo aplicó el siguiente proceso de control de cambios ágil: 1. **Identificación y Registro:** Toda solicitud de cambio se ingresó en el backlog de tareas de **TODO.md**. 2. **Evaluación de Impacto técnico y de tiempo:** El Scrum Master y los desarrolladores analizaron el impacto (ver tabla de impacto) y determinaron si el cambio afectaba la fecha de defensa del 22 de junio. 3. **Aprobación del Product Owner:** Se consultó con los profesores la viabilidad de los cambios (por ejemplo, validar el reemplazo del pago con tarjeta por transferencias directas). 4. **Implementación en Rama Aislada (feature/):** El código modificado se desarrolló en ramas secundarias de Git. 5. **Verificación y Fusión:** La suite de pruebas de Vitest certificó que el cambio no generara regresiones de código antes de hacer merge a la rama **develop** y finalmente a **main**.

24. Seguimiento y Control del Proyecto

24.1 Estado Actual del Proyecto

El proyecto se encuentra en el estado final de la **Etapla 4**, alineado estrictamente con el historial de versiones acumulado y el backlog de **TODO.md**:

- **Funcionalidades Finalizadas:**
- Configuración del monorrepo con **pnpm** y entorno de bases de datos PostgreSQL y MongoDB locales en Docker.
- Autenticación segura JWT, encriptación bcrypt de contraseñas y redirección post-login fluida.
- API de perfiles de asociados y autogestión de datos en Libro Registro de Asociados.
- Orquestación de datos síncrona (Data Mashup) para el Hero de Campañas destacadas.
- Checkout de donaciones declarando transferencias y panel de administración interactivo para aprobar/rechazar transferencias sin doble clic.

- Control transaccional de concurrencia mediante bloqueo exclusivo de fila **SELECT FOR UPDATE** en PostgreSQL.
- Validación contable de límites financieros en la API de donaciones.
- Gestor de noticias en MongoDB con sanitización DOMPurify en el frontend.
- Envío automático de agradecimientos por email transaccional SMTP.
- Middlewares de rate limiting por IP y sanitización contra inyecciones NoSQL.
- Suite de 47 pruebas de integración automatizadas en Vitest con base de test aislada y limpia.
- **Funcionalidades en Progreso:**
 - Pruebas finales de la compilación PDF del Documento Integral de Gestión de Software.
- **Funcionalidades Pendientes (Backlog a futuro):**
 - Autovalidación automatizada de transferencias con la API bancaria de la cooperadora (requiere convenios legales y presupuestarios externos).

24.2 Indicadores de Avance

Para controlar la marcha del proyecto, se definieron los siguientes indicadores clave simples:

- **Porcentaje de Funcionalidades Completadas: 95%** (19 de las 20 tareas de la WBS finalizadas satisfactoriamente).
- **Velocidad del Sprint: 19.2 Story Points por Sprint** (Cumplimiento de los 77 Story Points estimados a lo largo de los 4 sprints de 2 semanas de desarrollo).
- **Hitos Alcanzados: 5 de 6** hitos aprobados y cerrados. Hito final (Defensa Oral) programado para el 22 de junio de 2026.
- **Cobertura de Pruebas (Test Coverage): 86.4%** de cobertura del código backend (47 casos de prueba de integración validados secuencialmente).

24.3 Riesgos Actuales (Vigencia)

- **R-01 (Concurrencia Contable): Mitigado.** El bloqueo exclusivo de fila **SELECT FOR UPDATE** y los tests de estrés integrados garantizan que el riesgo esté controlado en un 100%.
- **R-04 (Retraso de Tiempo): Bajo Control.** Con la totalidad del código integrado en **main** y las pruebas pasando con éxito, el riesgo de no llegar a la entrega es casi nulo.
- **R-05 (SMTP en Producción): Bajo Control.** Las credenciales SMTP transaccionales han sido parametrizadas y se validó el envío de correos HTML a través de cuentas seguras de aplicación.
- **R-10 (Donaciones Huérfanas): Bajo Control.** La lógica de negocio obliga al operador a contrastar los datos de la transferencia declarada en el panel contra su homebanking antes de dar la aprobación definitiva, impidiendo que declaraciones falsas alteren el acumulado real.

24.4 Acciones Correctivas

En caso de registrarse desvíos de alcance o problemas durante la fase final de despliegue local:

- **Ante fallos de concurrencia en bases virtuales locales:** Reiniciar contenedores Docker mediante **docker-compose down && docker-compose up -d** para restaurar conexiones limpias.
- **Ante retrasos en la redacción del manual de administración:** Congelar el desarrollo de funcionalidades complementarias del backlog a futuro (como la conciliación automática) y redistribuir el esfuerzo humano en pulir la documentación técnica final.

25. Conclusión General

El desarrollo y gestión de este portal web para la Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra" de Necochea demostró la importancia crítica de aplicar metodologías formales de ingeniería de software en proyectos reales.

En primer lugar, la **planificación sistemática** a través de la descomposición WBS y estimaciones relativas por Story Points permitió al equipo acotar el alcance, evitar desvíos temporales y garantizar que un desarrollo de alta complejidad técnica (que involucra dos motores de bases de datos diferentes operando en simultáneo) pudiera culminarse exitosamente dentro del rígido plazo del cuatrimestre universitario.

En segundo lugar, el análisis proactivo de **riesgos contables e informáticos** derivó en decisiones técnicas sumamente maduras. La implementación de un flujo de validación manual en el panel de administración, reforzado por transacciones SQL seguras y bloqueos exclusivos de concurrencia, clausuró la posibilidad de inconsistencias contables o fraude por declaraciones falsas, convirtiendo a la plataforma en un entorno seguro y transparente capaz de generar confianza en los vecinos donantes.

Por último, el **seguimiento constante** del backlog en **TODO.md** y la suite de Vitest de 47 pruebas automatizadas blindaron al portal contra regresiones y fallos imprevistos. Este proyecto no solo resolvió una necesidad administrativa real para la Cooperadora del Hospital Municipal —digitalizando y despapelando sus registros de socios y aportes—, sino que también representó un aprendizaje profesional de gran valor para los integrantes del equipo en la coordinación de equipos ágiles, la toma de decisiones técnicas fundamentadas y el aseguramiento de la calidad del software.